COSTE COMPUTACIONAL

Entornos de Desarrollo

Marco Valiente
DAW1

Contenido

Introducción 2

Desarrollo 3

Introducción

Con los conocimientos adquiridos, en esta práctica vamos a escoger un código el cual ya hayamos hecho, y vamos a calcular de este lo que es su coste computacional.

Desarrollo

El algoritmo que vamos a decidir analizar, va a ser de la práctica de DawBank. De este, vamos a ver el código de nuestra selección de cuenta bancaria. El código ejemplo es el siguiente:

```
public static int seleccionCuentaBancaria(Banco banco) {
        Scanner in;

        int IDcuenta = 0;
        boolean encontrada = false;

        String inputSeleccionCuenta;
        String resultado = "Seleccione una de sus Cuentas Bancarias:\n";
        int num = banco.getNumCuentasBancarias();
        StringBuilder colNums  = new StringBuilder("");

        for (int i = 0; i < num; i++) {
            resultado += i + ". " +
banco.obtenerCuentaBancaria(banco.getColeccionIBAN()[i]).getTitular().get
Nombre() +
                    " (" +
banco.obtenerCuentaBancaria(banco.getColeccionIBAN()[i]).getIBAN() +
")\n";
        }
        MyUtils.imprimir(resultado);

        for (int i = 0; i < num; i++) {
            colNums.append(i);
            if (i<num-1) {
                colNums.append("|");
            }
        }

        Pattern numForm = Pattern.compile("(" + colNums + ")");
        Matcher matcherIBAN;
        Matcher matcherNum;
        do {
            MyUtils.imprimirSinSalto("\nSeleccione el numero asociado a
esa cuenta o su IBAN: ");
            in = new Scanner(System.in);
            inputSeleccionCuenta = in.nextLine();
            matcherIBAN = IBAN_FORM.matcher(inputSeleccionCuenta);
            matcherNum = numForm.matcher(inputSeleccionCuenta);
            if (!matcherIBAN.matches() && !matcherNum.matches()) {
                MyUtils.imprimir("Input no Valido, intentalo de Nuevo");
            }
            if (matcherIBAN.matches()) {
                for (int i = 0; i < num; i++) {
```

```
                    if(banco.obtenerCuentaBancaria(banco.getColeccionIBAN
()[i]).getIBAN().equalsIgnoreCase(inputSeleccionCuenta)) {
                        IDcuenta = i;
                        encontrada = true;
                    }
                }
                if (!encontrada) {
                    MyUtils.imprimir("\nIBAN NO EXISTENTE, seleccionando
cuenta por defecto\n");
                }
            }
            if (matcherNum.matches()) {
                IDcuenta = Integer.parseInt(inputSeleccionCuenta);
            }
        } while (!matcherIBAN.matches() && !matcherNum.matches());

        MyUtils.imprimir("Su cuenta seleccionada ha sido:\n");
        MyUtils.imprimir(banco.obtenerCuentaBancaria(banco.getColeccionIB
AN()[IDcuenta]).toString());

        return IDcuenta;
    }
```

Este es un método que devuelve un INT de forma que se pueda seleccionar esa casilla dentro de un Array. Tenemos varias declaraciones O(cte) al comienzo del código, hasta que llegamos a nuestro primer bucle for. Dentro de este bucle, usamos la función obtenerCuentaBancaria, que podemos ver aquí, más adelante:

```
    //Coleccion
    private HashMap<String,CuentaBancaria> coleccionCuentasBancarias;
```

Declarado, y tomado tal que:

```
    public CuentaBancaria obtenerCuentaBancaria(String IBAN) {
        return this.coleccionCuentasBancarias.get(IBAN);
    }
```

Este es una búsqueda por IBAN de los objetos que aparecen dentro del HashMap, pero, cabe mencionar que, aunque si fuese un Array o un ArrayList, esta búsqueda sería un coste adicional de O(n), los HashMaps son diseñados con un sistema de Buckets que permite un acceso casi instantáneo independientemente del tamaño de los datos, dando al coste computacional de esta búsqueda, el valor de O(cte).

También estamos seleccionando de nuestra colección de IBAN, que, esta, en contraposición, es de coste O(cte), ya que es llamar a una posición específica de un Array.

El loop en sí, es, por tanto, un coste computacional de O(n), ya que num es de tamaño n.

Después de este, tenemos otro loop que construye el patrón de los inputs que se necesitan introducir dentro del Stringbuilder. Este es otra repetición de O(n) iteraciones, y esta en particular, se podría anidar con el bucle anterior para reducir de O(2n) a O(n).

Una ver creado e imprimido esto dentro de la pantalla, que es un coste computacional de alrededor O(2n+cte), podemos analizar el input, que, en este caso, es más sencilla. Pedir datos con patrón es un bucle, pero que es de una única iteración en caso de que estos datos sean dados de forma correcta, por lo que recibe O(cte) a pesar de ser un do-while. Obtener cuenta bancaria podría ser de nuevo otro coste O(n) si no fuese porque HashMap está optimizado para ser O(cte), y la siguiente iteración que nos cuesta O(n/2) es encontrar el IBAN dentro de nuestra selección. Al ser una búsqueda, este tiene un promedio de ser encontrado después de n/2 intentos, asumiendo un azar a la hora de realizar tanteos, que, al final, no es el caso.

Por último, una vez que se ha acabado esto, se selecciona la cuenta de forma definitiva, que sería de nuevo un O(n) si esta no fuese un HashMap. Después de esto, se envía este número de vuelta como finalización del algoritmo.

Se concluye, entonces, que, sumando todos los costes computacionales que se añaden, el resultado de esta operación es:

O(cte) + O(n) + O(n) + O(n/2) = O(5n/2 + cte)

Este es el coste final de la selección de IBAN, Es mejorable, en términos absolutos, ya que se puede reducir a ser un simple O(3n/2 + cte) si se unen el primer y segundo for que se realiza.